

C-CAT C Language

Complete Revision Guide

Zero to Exam Ready

#	Topic	C-CAT Weight
1	C Language Foundations - History, Keywords, Standards	~1-2 Qs
2	Data Types, Type Modifiers & Qualifiers	~1-2 Qs
3	Operators - All Types + Precedence & Associativity	Up to 3 Qs
4	Pointers - Declaration, Arithmetic, NULL/Void/Wild	Up to 3 Qs
5	Functions - Pass by Value/Ref, Recursion, Storage	~2 Qs
6	Arrays - 1D/2D, Address Calc, Pointer-Array Link	~2 Qs
7	Strings - Library Functions, Null Terminator	~1 Q
8	Structures & Unions	~1 Q
9	File Handling	~1 Q

Chapter 1 | C Language Foundations - History, Keywords & Standards

Brief History

Year	Event
1972	Dennis Ritchie created C at Bell Labs
1978	K&R; C - first book by Kernighan & Ritchie
1989/90	ANSI C / C89 / C90 - first official standard (C-CAT assumes this)
1999	C99 - added // comments, long long, stdint.h
2011	C11 - current modern standard

[TRAP] C-CAT assumes C89/C90 behaviour unless the question says C99 or C11.

32 Keywords in C89 - Know These Cold

Data Types	int char float double void short long unsigned signed
Control Flow	if else for while do switch case break continue return goto default
Storage Class	auto register static extern
Type Qualifier	const volatile
Structure/Union	struct union enum typedef
Operator/Misc	sizeof

[TRAP] bool is NOT a keyword in C89. It was added in C99 via .

[TRAP] C-CAT favourite: 'Which of these is NOT a keyword?' - watch for bool, then, include, main.

Chapter 2 | Data Types, Type Modifiers & Qualifiers

Basic Data Types

Type	Size (bytes)	Range (approx)	Format Specifier
char	1	-128 to 127	%c or %d
short	2	-32768 to 32767	%d
int	2 or 4 *	depends on system	%d
long	4 or 8	+/- 2.1 billion (4-byte)	%ld
long long	8	+/- 9.2 x 10 ¹⁸	%lld
float	4	~3.4x10 ⁻³⁸ to 10 ³⁸	%f
double	8	~1.7x10 ⁻³⁰⁸ to 10 ³⁰⁸	%lf
void	0	no value	-

[TRAP] int size is compiler/platform dependent. C-CAT often assumes 2 bytes (older systems).

[TRAP] 3.14 without 'f' suffix is a DOUBLE (8 bytes), not float. 3.14f is float (4 bytes).

Type Modifiers

- >> signed - allows negative and positive (default for int, char). Range: -128 to 127 for signed char.
- >> unsigned - only 0 and positive. Doubles positive range. unsigned char: 0 to 255.
- >> short - makes int smaller (guaranteed 2 bytes).
- >> long / long long - makes int larger.

[TRAP] unsigned overflow wraps around: 255+1=0, 0-1=255 for unsigned char. This is defined behaviour.

[TRAP] signed overflow is UNDEFINED behaviour. unsigned overflow is DEFINED (wraps around).

Type Qualifiers

const	volatile
Value cannot be changed after declaration	Tells compiler: do not optimise/cache this variable
Compiler enforces - error if you try to assign	Value may change from external source (hardware, interrupt)
Used for fixed values, constants	Used in embedded systems
Example: const int MAX = 100;	Example: volatile int sensorVal;

[TRAP] const int x=5 and #define X 5 are NOT the same. const has type, lives in memory, is type-checked. #define is preprocessor text substitution - no type, no memory.

[NOTE] volatile const together is VALID. It means: program cannot write to it, but hardware may change it.

ASCII Values to Remember

Character	ASCII Value
'A'	65
'Z'	90
'a'	97
'z'	122

'0'	48
'g'	57
'\0' (null)	0

[TRAP] char c = 'A'; printf("%d", c); prints 65, not A. %d prints ASCII value.

Chapter 3 | Operators - All Types, Precedence & Associativity

Arithmetic Operators

Op	Name	Example	Result	Key Rule
+	Addition	5+3	8	-
-	Subtraction	5-3	2	-
*	Multiplication	5*3	15	-
/	Division	5/2	2 [!]	Integer division - decimal dropped
%	Modulo	5%2	1	Remainder. Only integers. Sign = left operand.

[TRAP] 5/2 = 2 in integer division, NOT 2.5. For 2.5: use 5.0/2 or (float)5/2.

[TRAP] -7%3 = -1, NOT 2. Sign of modulo result = sign of LEFT operand.

Increment & Decrement - Most Tested in C-CAT

++x (Pre-increment)	x++ (Post-increment)
Increment x FIRST	Use current value FIRST
Then use the new value	Then increment x
int b = ++a; // a=6,b=6 (if a was 5)	int y = x++; // y=5,x=6 (if x was 5)

[TRAP] printf("%d %d", x++, x++) - modifying x twice between same sequence points = UNDEFINED BEHAVIOUR. Answer is always D) Undefined Behavior.

[TRAP] int b = a++ + ++a; - same pattern, same answer: Undefined Behaviour.

True / False in C - Critical Foundation

>> C has NO true/false type in C89.

>> 0 = false. ANY non-zero value = true (positive, negative, -1, -99 - all true).

>> Relational and logical operators always return exactly 0 or 1.

>> printf("%d", 5>3) prints 1. printf("%d", 5<3) prints 0. Never prints 'true'/'false'.

[TRAP] -1 is non-zero, so -1 is TRUE in C. Only 0 is false.

Relational Operators

Op	Meaning	Example	Result
==	Equal to	5==5	1
!=	Not equal to	5!=3	1
>	Greater than	5>3	1
<	Less than	5<3	0
>=	Greater or equal	5>=5	1
<=	Less or equal	3<=5	1

[TRAP] = is assignment. == is comparison. if(x=5) always true (assigns 5, non-zero). if(x==5) checks equality.

Logical Operators & Short-Circuit Evaluation

Op	Name	Rule	Short-circuits when
&&	Logical AND	Both must be non-zero	Left side is 0 (false) - right side SKIPPED
	Logical OR	At least one non-zero	Left side is non-zero (true) - right side SKIPPED
!	Logical NOT	Flips 0<->non-zero	-

[TRAP] if(a || b++) - if a is true (non-zero), b++ is NEVER executed. b stays unchanged.

[TRAP] Short-circuit protects: (x!=0) && (10/x > 1) - right side never runs if x==0, so no division by zero.

Bitwise Operators

Op	Name	Rule	5 op 3	Result
AND	Bitwise AND	1 only if BOTH are 1	101 AND 011	001 = 1
OR	Bitwise OR	1 if EITHER is 1	101 OR 011	111 = 7
XOR	Bitwise XOR	1 if DIFFERENT	101 XOR 011	110 = 6
NOT(~)	Bitwise NOT	Flips all bits	~5	-6 (~x = -(x+1))
LS(<<)	Left Shift	$x \text{ LS } n = x * 2^n$	5 LS 1	10
RS(>>)	Right Shift	$x \text{ RS } n = x / 2^n$	20 RS 2	5

[TRAP] ~5 = -6, NOT -5. Formula: $\sim x = -(x+1)$. $\sim 0 = -1$. $\sim(-1) = 0$.

[TRAP] $4 \& 2 = 0$ (no matching bits: 100 & 010 = 000) BUT $4 \&\& 2 = 1$ (both non-zero = true). COMPLETELY DIFFERENT.

[TRAP] $x \& 1 == 0$ does NOT check if x is even. Precedence: == runs before &. So it means $x \& (1 == 0) = x \& 0 = 0$ ALWAYS. Write $(x \& 1) == 0$ to check even.

Ternary Operator

condition ? value_if_true : value_if_false

>> Only ONE branch executes - the chosen one. The other branch's side effects NEVER happen.

>> int max = (a > b) ? a : b;

>> Nested: $x > 0 ? 1 : x < 0 ? -1 : 0$ (sign function)

[TRAP] int r = (a > 3) ? a++ : b++; - if a > 3, only a++ runs. b++ never executes.

sizeof Operator

>> Returns size in BYTES of a type or variable.

>> Evaluated at COMPILE TIME - the expression inside is NEVER executed.

>> sizeof('hello') = 6 (5 chars + 1 null terminator \0)

>> sizeof(3.14) = 8 (double, not float - no 'f' suffix)

>> sizeof(int*) = 4 on 32-bit, 8 on 64-bit system

[TRAP] sizeof(x++) does NOT increment x. sizeof never executes its operand - only checks its type.

[TRAP] sizeof('hello') = 6 (includes \0). strlen('hello') = 5 (excludes \0). Difference is always 1.

Comma Operator

>> Evaluates LEFT side first (sequence point), then RIGHT side.

>> Result of whole expression = value of RIGHTMOST expression.

>> int x = (3, 5, 7); -> x = 7

>> a[i=1, i+1] -> i=1 first, then i+1=2, so a[2]

[TRAP] Comma between FUNCTION ARGUMENTS is NOT the comma operator - no sequence point. Comma INSIDE [] or () as expression IS the comma operator - sequence point guaranteed.

Operator Precedence & Associativity - Full Table

Priority	Operators	Category	Associativity
1 - Highest	() [] . ->	Function call, array, member	Left -> Right
2	++ -- (postfix) ! ~ + - (unary) * & (unary) sizeof	Unary operators	Right -> Left
3	* / %	Multiplicative	Left -> Right
4	+ -	Additive	Left -> Right
5	<< >>	Bitwise shift	Left -> Right
6	< <= > >=	Relational	Left -> Right
7	== !=	Equality	Left -> Right
8	&	Bitwise AND	Left -> Right
9	^	Bitwise XOR	Left -> Right
10		Bitwise OR	Left -> Right
11	&&	Logical AND	Left -> Right
12		Logical OR	Left -> Right
13	?:	Ternary	Right -> Left
14	= += -= *= /= %= etc.	Assignment	Right -> Left
15 - Lowest	,	Comma operator	Left -> Right

[TRAP] == and != are LOWER priority than < > <= >=. So `2<3==1` means `(2<3)==1 -> 1==1 -> 1`.

[TRAP] Bitwise & | ^ are LOWER than ==. So `x&1==0` means `x&(1==0) = x&0 = 0` always.

[TRAP] Assignment is Right-to-Left. `a=b=c=5 -> c=5` first, then `b=5`, then `a=5`.

[TRAP] Comma has LOWEST priority of all operators.

Chapter 4 | Pointers - Declaration, Arithmetic, NULL/Void/Wild

Memory Model - the Foundation

- >> Every variable occupies memory. That memory has an ADDRESS (a number identifying its location).
- >> int x=10 at address 2000: the box at 2000 holds value 10.
- >> &x; gives the address (2000). *p (after p=&x;) gives the value at that address (10).

The Three Core Operations

int *p	Declaration - p is a pointer that will store the address of an int
p = &x;	Address-of - store the address of x into p
*p	Dereference - go to the address in p and get/set the value there

```
int x = 10;
int *p = &x;      // p stores address of x (e.g. 2000)
printf("%d", *p); // 10 - value at address 2000
*p = 99;          // x is now 99 - changed through pointer
printf("%d", x);  // 99
```

[TRAP] int *p = &x; stores the ADDRESS of x in p. int *p = x stores the VALUE 10 as if it were an address - DANGEROUS wild pointer.

[TRAP] *p = 99 changes the VALUE at address. p = 99 makes p point to address 99 - random dangerous memory.

[TRAP] int *p, q - only p is a pointer. q is a plain int. To make both pointers: int *p, *q.

Pointer Arithmetic

- >> Adding 1 to a pointer moves it forward by sizeof(type) bytes - NOT by 1 byte.
- >> int* p+1 -> moves 4 bytes forward.
- >> char* p+1 -> moves 1 byte forward.
- >> double* p+1 -> moves 8 bytes forward.

```
int arr[4] = {10,20,30,40}; // arr at address 2000
int *p = arr;              // p = 2000
printf("%d", *(p+2));      // 30 - address 2008
```

[TRAP] arr[i] == *(arr+i) == *(i+arr) == i[arr]. All four are identical. 2[arr] is valid C and gives arr[2].

[TRAP] arr is a CONSTANT - arr++ is a COMPILE ERROR. A pointer int *p=arr; p++; is fine.

Array Name = Address of First Element

```
int arr[3] = {10,20,30};
arr        == &arr[0] // same address
*arr       == arr[0]  // same value = 10
*(arr+i)   == arr[i]  // always equivalent
```

NULL, void, and Wild Pointers

NULL Pointer	void Pointer
int *p = NULL;	void *p;
Value = 0 (macro defined as 0)	No fixed type - universal pointer
Points to nothing - intentionally empty	Can point to any type without cast
if(p) is FALSE when p is NULL	CANNOT be dereferenced directly
Dereferencing NULL = segfault (runtime crash)	Must cast first: *(int*)p

Always check before dereferencing	CANNOT do pointer arithmetic directly
-----------------------------------	---------------------------------------

[TRAP] NULL != uninitialised. NULL is deliberate and safe to check. Wild pointer is accidental garbage.

[TRAP] Dereferencing NULL = runtime crash (segfault). But code COMPILES fine. Crash is at runtime, not compile.

[TRAP] void* can receive any pointer without cast. But to dereference: must cast. *(void*) is compile error.

Wild Pointer

- >> A pointer declared but never initialised - holds garbage address.
- >> Dereferencing it = undefined behaviour - likely segfault or silent corruption.
- >> Code COMPILES fine. Problem only appears at runtime.
- >> Fix: always initialise to NULL or a valid address immediately.

[TRAP] Wild pointer dereferencing = RUNTIME error, NOT compile error. C-CAT favourite question.

sizeof Pointer

32-bit system	ALL pointers = 4 bytes (int*, char*, double*, void* - all same)
64-bit system	ALL pointers = 8 bytes (int*, char*, double*, void* - all same)
sizeof(int*) vs sizeof(int)	On 64-bit: sizeof(int*)=8, sizeof(int)=4. Pointer > data type.

[TRAP] Pointer size NEVER depends on what it points to. char* and double* are the same size on the same system.

Double Pointer

```
int x = 10;
int *p = &x;    // p stores address of x
int **pp = &p;  // pp stores address of p
// *pp = p (address of x)
// **pp = x = 10
```

Chapter 5 | Functions - Pass by Value/Reference, Recursion, Storage Classes

Function Anatomy

```
return_type function_name(parameter_list) {  
    /* body */  
    return value;  
}
```

Parameter	Variable in function DEFINITION - placeholder. int add(int a, int b) -> a,b are parameters
Argument	Actual value passed in function CALL. add(3,4) -> 3,4 are arguments
void func	Returns nothing. CANNOT be used in expression: int x = voidFunc(); is ERROR

Pass by Value vs Pass by Reference

Pass by Value	Pass by Reference (via pointer)
void modify(int n)	void modify(int *n)
modify(x) - passes VALUE	modify(&x;) - passes ADDRESS
A COPY is made inside function	Function receives address of original
Original variable UNCHANGED	Original variable CAN be changed
n=99 inside does NOT change x	*n=99 inside DOES change x

```
// PASS BY VALUE - x unchanged  
void modify(int n) { n = 99; }  
int x=10; modify(x); printf("%d",x); // 10  
  
// PASS BY REFERENCE - x changed  
void modify(int *n) { *n = 99; }  
int x=10; modify(&x); printf("%d",x); // 99
```

[TRAP] If you see `modify(int n)` and question asks if `x` changes - `x` NEVER changes. Always pick 'unchanged'.

Recursion

- >> A function that calls itself.
- >> MUST have a BASE CASE - condition that stops recursion.
- >> Without base case -> infinite calls -> stack overflow (RUNTIME crash).
- >> Each call adds a new frame to the call stack.

```
int factorial(int n) {  
    if (n == 0) return 1;    // BASE CASE  
    return n * factorial(n-1); // RECURSIVE CASE  
}  
  
// factorial(4) = 4*3*2*1 = 24
```

[TRAP] `printf` BEFORE recursive call -> prints in ORIGINAL order (3 2 1).

[TRAP] `printf` AFTER recursive call -> prints in REVERSE order (1 2 3). Single line position = completely different output.

```
void f(int n) {  
    if(n==0) return;  
    printf("%d ",n);  
    f(n-1);  
}  
  
void f(int n) {  
    if(n==0) return;  
    f(n-1);  
    printf("%d ",n);  
}
```

```
} // output: 3 2 1    } // output: 1 2 3
```

Storage Classes - All 4

Class	Default Value	Where Stored	Lifetime	Key Rule
auto	Garbage	Stack	Block exit	Default for local vars. Never zero-init.
register	Garbage	CPU register	Block exit	Cannot use & on register variable.
static	0 (zero)	Data segment	Entire program	Init runs ONCE. Value persists between calls.
extern	0 (if global)	Data segment	Entire program	Declaration only - variable defined elsewhere.

[TRAP] auto variables are NOT zero-initialised - they have garbage values. Only static and global are zero-init.

[TRAP] static inside a function: initialised once, retains value between calls. int x=0 runs only on first call.

[TRAP] register variable - compiler may ignore request. Cannot take address with &.

[TRAP] extern int x; is DECLARATION (no memory). The actual definition must exist elsewhere.

Chapter 6 | Arrays - 1D/2D, Address Calculation, Pointer-Array Link

1D Array Basics

```
int arr[5] = {10,20,30,40,50};
// Index: 0   1   2   3   4
arr[0] = 10  (first)
arr[4] = 50  (last)
arr[5]      // OUT OF BOUNDS - undefined behaviour, no compile error
```

[TRAP] Array index starts at 0. Last valid index = size-1. arr[5] for size-5 array = UB.

Address Calculation Formula

Formula	address of arr[i] = base_address + (i x sizeof(type))
Example (int)	base=1000, arr[2] = 1000 + (2x4) = 1008
Example (char)	base=1000, arr[2] = 1000 + (2x1) = 1002
Example (double)	base=1000, arr[2] = 1000 + (2x8) = 1016

[TRAP] Jumps are by sizeof(type) bytes, NOT by 1 byte.

Pointer-Array Equivalence

```
int arr[3] = {10,20,30};
arr      == &arr[0]      // same address
*arr     == arr[0] == 10  // same value
arr[i]   == *(arr+i)     // always equivalent
*(arr+2) == arr[2] == 30
2[arr]   == arr[2] == 30 // valid C - addition is commutative
```

[TRAP] arr++ is COMPILE ERROR - array name is a constant address. int *p=arr; p++; is fine.

[TRAP] *arr+2 != *(arr+2). Precedence: * runs before +. *arr+2 = (*arr)+2 = 10+2 = 12. *(arr+2) = 30.

sizeof with Arrays

```
int arr[5] = {1,2,3,4,5};
sizeof(arr)           // 20 = 5 elements x 4 bytes
sizeof(arr)/sizeof(arr[0]) // 5 - number of elements (standard idiom)
// INSIDE A FUNCTION:
void func(int arr[]) {
    sizeof(arr); // gives POINTER SIZE (4 or 8), NOT array size!
}
```

[TRAP] When array is passed to a function, it DECAYS to a pointer. sizeof inside function gives pointer size.

2D Arrays

```
int a[2][3] = {{1,2,3},{4,5,6}};
// Stored row-major: 1 2 3 4 5 6 (contiguous in memory)
// Address of a[i][j] = base + (i*COLS + j) * sizeof(type)
// a[1][2] = base + (1*3+2)*4 = base+20 -> value=6
a[i][j] == (*(a+i)+j) // pointer equivalent
```

[TRAP] 2D array is stored row by row in memory (row-major order). a+i jumps by COLS*sizeof(type) (one full row).

Chapter 7 | Strings - Library Functions, Null Terminator, Pointer Access

String Fundamentals

>> A string in C = array of char ending with null terminator \0 (ASCII value 0).

>> "hello" occupies 6 bytes in memory: h e l l o \0

>> \0 is NOT the character '0'. \0 has ASCII value 0. '0' has ASCII value 48.

```
char str[] = "hello";    // str = {'h','e','l','l','o','\0'}
sizeof(str) = 6          // includes \0
strlen(str) = 5           // excludes \0 - counts visible chars only
```

[TRAP] sizeof(string) = length+1 (includes \0). strlen(string) = length (excludes \0). Difference is always 1.

String Library Functions - #include

Function	What it does	Key Trap
strlen(s)	Returns length - excludes \0	Returns size_t (unsigned). strlen("") = 0
strcpy(dest,src)	Copies src into dest including \0	dest must be large enough
strcat(dest,src)	Appends src to end of dest	dest must have enough space for both
strcmp(s1,s2)	Compares s1 and s2	Returns 0 if EQUAL, <0 or >0 if not
strchr(s,c)	Finds first occurrence of char c	Returns pointer to char, NULL if not found
strstr(s1,s2)	Finds s2 in s1	Returns pointer to match, NULL if not found

[TRAP] strcmp returns 0 for EQUAL strings - not 1. if(strcmp(s1,s2)) is TRUE when strings are DIFFERENT.

[TRAP] strcpy copies including \0. If dest is too small -> buffer overflow (undefined behaviour).

Character Array vs String Pointer

```
char arr[] = "hello";    // array - modifiable, stored on stack
char *p = "hello";       // pointer to string LITERAL - read-only
*p = 'H';                 // UNDEFINED BEHAVIOUR - string literal is read-only
```

[TRAP] String literals are read-only. Modifying via pointer = UB. Use char arr[] for modifiable strings.

String Access via Pointer

```
char str[] = "hello";
char *p = str;
printf("%c", *p);        // h
printf("%c", *(p+1));    // e
printf("%s", p);          // hello - %s prints from p until \0
printf("%s", p+2);        // llo - starts printing from index 2
```

Chapter 8 | Structures & Unions

Structures

```
struct Student {  
    int roll;  
    char name[20];  
    float marks;  
};  
struct Student s1;           // declare variable  
struct Student arr[3];       // array of structs
```

Dot vs Arrow Operator

Dot Operator (.)	Arrow Operator (->)
Used with normal struct variable	Used with POINTER to struct
struct Student s1;	struct Student *p = &s1;
s1.roll = 10;	p->roll = 10;
s1.marks = 95.5;	(*p).roll = 10; // same thing

[TRAP] p.roll when p is a pointer = **COMPILE ERROR**. Must use p->roll or (*p).roll.

[TRAP] p->roll is **SHORTHAND** for (*p).roll. They are identical.

sizeof struct - Padding

```
struct A { char c; int i; };  
sizeof(struct A) = 8 // NOT 5 (1+4)  
// Compiler adds padding: char(1) + padding(3) + int(4) = 8  
// Rule: members aligned to their own size boundary
```

[TRAP] sizeof(struct) is **NOT** simply the sum of member sizes. Padding/alignment makes it larger.

Bit Fields

```
struct Flags {  
    unsigned int isActive : 1; // 1 bit - values 0 or 1  
    unsigned int type      : 3; // 3 bits - values 0 to 7  
};
```

[TRAP] Cannot use & (address-of) on a bit field member - compile error.

Unions - Shared Memory

```
union Data {  
    int i;           // 4 bytes  
    float f;         // 4 bytes  
    char c;          // 1 byte  
};  
// sizeof(union Data) = 4 (size of LARGEST member)  
// All members share the SAME memory location
```

struct - own memory	union - shared memory
Each member has its own space	All members share ONE memory location
sizeof = sum of members + padding	sizeof = size of LARGEST member
All members can hold values simultaneously	Only ONE member holds valid value at a time

struct {int i; char c;} -> sizeof = 8 (padded)	union {int i; char c;} -> sizeof = 4
--	--------------------------------------

[TRAP] Writing to one union member **OVERWRITES** all others - they share the same bytes.

[TRAP] union Data d; d.i=65; printf('%c',d.c); prints 'A' - same memory, reinterpreted as char.

Chapter 9 | File Handling - Modes, Functions, Sequential & Random Access

Opening and Closing Files

```
FILE *fp;
fp = fopen("data.txt", "r"); // open file
if (fp == NULL) {           // ALWAYS check
    printf("Error");
}
fclose(fp);                 // ALWAYS close
```

[TRAP] fopen returns NULL if file doesn't exist (in 'r' mode). Using NULL fp = crash. Always check.

[TRAP] Always call fclose() after done - prevents data loss and resource leaks.

File Modes - Memorise All 6

Mode	Read?	Write?	File must exist?	Behaviour
"r"	Yes	No	YES	Read only. Error if file missing.
"w"	No	Yes	No	Write only. Creates file. ERASES existing content.
"a"	No	Yes	No	Append. Creates if not exists. Writes at END.
"r+"	Yes	Yes	YES	Read+Write. Error if file missing.
"w+"	Yes	Yes	No	Read+Write. Creates file. ERASES existing content.
"a+"	Yes	Yes	No	Read+Append. Creates if not exists.

[TRAP] "w" mode ERASES the file immediately on opening - even before you write a single byte.

[TRAP] "r" mode fails (returns NULL) if file does not exist. "w" and "a" create the file if missing.

File I/O Functions

Function	What it does
fprintf(fp, "%d", x)	Write formatted data to file (like printf but to file)
fscanf(fp, "%d", &x)	Read formatted data from file (like scanf but from file)
fputc(ch, fp)	Write single character to file
fgetc(fp)	Read single character from file. Returns EOF at end.
fputs(str, fp)	Write string to file
fgets(str, n, fp)	Read string from file (reads at most n-1 chars + \0)
fclose(fp)	Close file - ALWAYS call this

Sequential vs Random Access

Sequential Access	Random Access
Read/write from start to end in order	Jump to any position in file
Default behaviour - pointer moves forward	Use fseek() to move file pointer
fgetc, fputc, fprintf, fscanf	Then read/write from that position
Like reading a book page by page	Like jumping to any page directly


```
fseek(fp, 0, SEEK_SET);    // go to beginning of file
fseek(fp, 0, SEEK_END);    // go to end of file
fseek(fp, n, SEEK_CUR);    // move n bytes from current position
ftell(fp);                // returns current position (bytes from start)
rewind(fp);                // go back to beginning (same as fseek(fp,0,SEEK_SET))
```

[TRAP] fseek(fp, n, SEEK_CUR) - n can be negative (move backwards).

Chapter 0 | Quick Reference - Most Tested C-CAT Traps

Output = 0 or Undefined Behaviour Checklist

- [!] `x++` and `x++` in same `printf` -> UB (modify same var twice, no sequence point)
- [!] `a++ + ++a` in one expression -> UB
- [!] `int *p = x` (value not address) -> wild pointer -> runtime crash
- [!] Dereferencing `NULL` -> segfault at runtime (compiles fine)
- [!] Dereferencing wild pointer -> UB / segfault at runtime
- [!] `*void_ptr` directly -> compile error (must cast first)
- [!] `arr++` -> compile error (array name is constant)
- [!] `®ister;_var` -> compile error (no address for register)
- [!] `sizeof(x++)` -> `x` never incremented (sizeof doesn't execute operand)
- [!] `x&1==0` -> always 0 (`==` runs before `&`) - write `(x&1)==0` to check even
- [!] `5/2 = 2` not 2.5 (integer division truncates)
- [!] `-7%3 = -1` not 2 (modulo sign follows left operand)
- [!] `~5 = -6` not -5 (formula: `~x = -(x+1)`)
- [!] `4&2 = 0` but `4&&2 = 1` (bitwise vs logical AND)
- [!] `strcmp` equal strings returns 0 (not 1)
- [!] `sizeof('hello') = 6` not 5 (includes `\0`)
- [!] `strlen` gives count without `\0`. `sizeof` gives bytes including `\0`.
- [!] `sizeof(struct)` has padding - NOT sum of members
- [!] `sizeof(union)` = size of LARGEST member
- [!] Writing to union member overwrites all others (shared memory)
- [!] 'w' mode erases file on open immediately
- [!] `fopen` returns `NULL` if file missing in 'r' mode - always check
- [!] Array passed to function decays to pointer - `sizeof` gives pointer size inside function
- [!] `printf` BEFORE recursive call: original order. AFTER: reverse order.
- [!] auto variables have garbage values (NOT zero-initialised)
- [!] static variable: init runs ONCE. Value persists between function calls.
- [!] `NULL` = deliberate zero. Wild = accidental garbage. Never confuse them.
- [!] `p->member` shorthand for `(*p).member`. `p.member` on pointer = compile error.

Format Specifiers Quick Reference

Type	Format	Type	Format
int	%d	unsigned int	%u
long	%ld	long long	%lld
float	%f	double	%lf
char	%c	string	%s
pointer	%p	hex	%x

[NOTE] You have covered ALL 9 topic areas. Trust your preparation. Identify the trap first, then answer.

